

CIF Microarchitecture Specification

Block: Response Tracker

CIF Design Team

June 19, 2026

Contents

1	Overview	2
2	Design Parameters	2
3	Internal Blocks	2
3.1	Register File	2
3.2	Alloc Write Logic	3
3.3	Completion Update Logic	3
3.4	Compare Logic	3
3.5	Head Pointer Array	3
3.6	Emit Logic	4
3.7	Free Logic	4
4	Interface Signals	4
4.1	Inputs	4
4.2	Outputs	5
4.3	Internal Signals	5
5	Functional Behaviour	5
5.1	Allocation Flow	5
5.2	Completion Flow	5
5.3	Emit Flow	6
5.4	Free Flow	6
6	Timing Notes	6
7	Edge Cases	6

1 Overview

The Response Tracker is a write-path block responsible for aggregating per-packet MC Core completions into a single AXI B channel response per transaction. It tracks every outstanding write transaction from tag allocation to final BRESP emission.

Its responsibilities are:

- Store per-transaction metadata at allocation time: expected packet count, completed packet count, error status.
- Increment the completed packet counter on each `WR_ACK` completion received from MC Core.
- Assert `txn_done` when all expected packets for a transaction have completed.
- Enforce AXI4 B channel ordering: BRESP is emitted in-order per AXID using a per-AXID head pointer.
- Propagate error status: if any packet in a burst returns an error completion, the sticky error bit is set and the transaction generates `BRESP = DECERR`.
- Free the occupied Tag Allocator slot on BRESP emission via the `WR_FREE` port.

Note on abstraction levels. The full CIF pipeline diagram shows MC Core as the sole input to the Response Tracker. At the internal logic level, the Tag Allocator also provides an allocation signal — this is a resource management path (writing `exp_pkts` at tag assignment time) that is separate from the data completion path. Both are correct; they operate at different abstraction levels.

2 Design Parameters

Parameter	Value	Description
<code>AXID_W</code>	6	AXI ID width; 64 unique IDs
<code>SEQNUM_W</code>	2	Seqnum width
<code>TXN_TAG_W</code>	8	Tag width = <code>AXID_W</code> + <code>SEQNUM_W</code>
<code>RT_DEPTH</code>	256	Register file entries = 2^8
<code>EXP_PKTS_W</code>	8	Expected packets width (covers <code>AWLEN+1</code> , max 256)
<code>CMP_PKTS_W</code>	8	Completed packets counter width
<code>N_AXIDS</code>	64	Head pointer array depth
<code>HEAD_W</code>	8	Head pointer width = <code>TXN_TAG_W</code>

Table 1: Response Tracker parameters

3 Internal Blocks

3.1 Register File

- **Structure:** 256-entry register file. Each entry stores:
 - `exp_pkts[7:0]` — expected packet count (= `AWLEN+1`)
 - `cmp_pkts[7:0]` — completed packet count
 - `status[3:0]` — MC Core status code from most recent error completion

- `err_sticky[0]` — sticky error bit; set on first error completion, never cleared until entry is freed
- **Addressing:** `addr[7:0] = TXN_TAG[7:0] = {AXID[5:0], seqnum[1:0]}`. Tag directly indexes the register file — no translation required.
- **Write ports:**
 - Alloc Write Logic: writes `exp_pkts`, initialises `cmp_pkts = 0`, `err_sticky = 0` on allocation.
 - Completion Update Logic: increments `cmp_pkts`, optionally sets `err_sticky` and `status` on each `WR_ACK` completion.
 - Free Logic: clears the entry on `BRESP` emission.
- **Read ports:** Continuously exposes `cmp_pkts` and `exp_pkts` to Compare Logic; exposes `err_sticky` and `status` to Emit Logic.

3.2 Alloc Write Logic

- Activated on `TAG_VALID` from the Tag Allocator.
- Receives `TXN_TAG[7:0]` and `AWLEN[7:0]`.
- Computes `exp_pkts = AWLEN + 1`.
- Issues `write_en` to the Register File at address `TXN_TAG[7:0]`, writing `exp_pkts` and initialising `cmp_pkts = 0`, `err_sticky = 0`.

3.3 Completion Update Logic

- Taps the MC Core completion bus: `RESP_VALID`, `resp_type[1:0]`, `tag[7:0]`, `status[3:0]`.
- **Filter:** acts only when `resp_type == 2'b01` (`WR_ACK`). `RD_DATA` completions (`resp_type == 2'b00`) are destined for the ROB and RTT and are ignored here.
- On a normal `WR_ACK` (`resp_type == 2'b01`, no error): asserts `inc_en` to the Register File at address `tag[7:0]`, incrementing `cmp_pkts` by 1.
- On an error completion (`resp_type == 2'b10`): increments `cmp_pkts` and additionally sets `err_sticky = 1` and latches `status[3:0]`. The sticky bit is never cleared by subsequent completions — one error poisons the entire transaction.

3.4 Compare Logic

- Purely combinational.
- Reads `cmp_pkts[7:0]` and `exp_pkts[7:0]` from the Register File for the entry currently being updated.
- Asserts `txn_done` as a one-cycle pulse when `cmp_pkts == exp_pkts`.
- `txn_done` is forwarded to Emit Logic to trigger `BRESP` evaluation.

3.5 Head Pointer Array

- **Structure:** 64×8 -bit registers, one per AXID. Each register holds the `TXN_TAG` of the next transaction expected to emit `BRESP` for that AXID.

- **Init:** Written on first allocation for an AXID (occupancy 0→1). On subsequent allocations for the same AXID, the head pointer is not updated — it is owned by Emit Logic from that point forward.
- **Advance:** Emit Logic increments the head pointer (mod 4 on the seqnum field) after each BRESP is emitted for that AXID.
- **Purpose:** Enforces AXI4 B channel ordering — BRESP is only emitted for the transaction at the head, not for any later-completing transaction of the same AXID.

3.6 Emit Logic

- Checks whether the transaction at `head_tag[AXID]` is done: reads `txn_done` from Compare Logic and confirms the completing tag matches the current head pointer for that AXID.
- If head slot is done:
 - Reads `err_sticky` and `status` from the Register File.
 - Asserts `BRESP_VALID` to the B Response Generator.
 - Drives `BID[5:0] = head_tag[7:2]` (AXID recovered from tag).
 - Drives `BRESP[1:0] = OKAY` if `err_sticky == 0`; `DECERR` if `err_sticky == 1`.
 - Asserts `advance_head` to the Head Pointer Array for the relevant AXID.
 - Asserts `emit_done` with `tag[7:0]` to Free Logic.
- If head slot is not yet done: Emit Logic holds. Later-completing transactions for the same AXID are buffered in the Register File until the head slot completes.

3.7 Free Logic

- Activated by `emit_done` from Emit Logic.
- Asserts `clear_en` to the Register File at address `tag[7:0]`, clearing the entry in the same cycle as BRESP emission.
- Simultaneously asserts `WR_FREE_VALID` and drives `WR_FREE_TAG[7:0]` to the Tag Allocator's `WR_FREE` port, decrementing the occupancy counter for the freed AXID.

4 Interface Signals

4.1 Inputs

Signal	Width	Source	Description
<code>TXN_TAG[7:0]</code>	8	Tag Allocator	Allocated tag at write transaction time
<code>AWLEN[7:0]</code>	8	Tag Allocator	Burst length; <code>exp_pkts = AWLEN+1</code>
<code>TAG_VALID</code>	1	Tag Allocator	Allocation strobe
<code>RESP_VALID</code>	1	MC Core	Completion valid
<code>resp_type[1:0]</code>	2	MC Core	00=RD_DATA (ignored), 01=WR_ACK, 10=ERR
<code>tag[7:0]</code>	8	MC Core	Completion tag; used as register file address
<code>status[3:0]</code>	4	MC Core	Error status; latched on error completion

Table 2: Response Tracker input signals

4.2 Outputs

Signal	Width	Destination	Description
BRESP_VALID	1	B Response Generator	BRESP ready to emit
BID[5:0]	6	B Response Generator	AXI ID recovered from <code>head_tag[7:2]</code>
BRESP[1:0]	2	B Response Generator	OKAY or DECERR
WR_FREE_VALID	1	Tag Allocator (WR_FREE port)	Free strobe; coincident with BRESP emit
WR_FREE_TAG[7:0]	8	Tag Allocator (WR_FREE port)	Tag being freed

Table 3: Response Tracker output signals

4.3 Internal Signals

Signal	Width	From	To	Description
<code>write_en</code>	1	Alloc Write Logic	Register File	Write strobe on allocation
<code>wr_addr[7:0]</code>	8	Alloc Write Logic	Register File	= <code>TXN_TAG[7:0]</code>
<code>exp_pkts[7:0]</code>	8	Alloc Write Logic	Register File	= <code>AWLEN+1</code>
<code>inc_en</code>	1	Completion Update Logic	Register File	Increment <code>cmp_pkts</code> strobe
<code>err_set</code>	1	Completion Update Logic	Register File	Set <code>err_sticky</code> on error
<code>inc_addr[7:0]</code>	8	Completion Update Logic	Register File	= <code>tag[7:0]</code> from MC Core
<code>cmp_pkts[7:0]</code>	8	Register File	Compare Logic	Current completed count
<code>exp_pkts[7:0]</code>	8	Register File	Compare Logic	Expected count (read port)
<code>err_sticky</code>	1	Register File	Emit Logic	Sticky error flag
<code>txn_done</code>	1	Compare Logic	Emit Logic	Pulse: all packets completed
<code>head_tag[7:0]</code>	8	Head Pointer Array	Emit Logic	Next tag to emit per AXID
<code>advance_head</code>	1	Emit Logic	Head Pointer Array	Increment head pointer for A
<code>emit_done</code>	1	Emit Logic	Free Logic	BRESP emitted; trigger free
<code>clear_en</code>	1	Free Logic	Register File	Clear entry strobe
<code>clear_addr[7:0]</code>	8	Free Logic	Register File	= tag of emitted transaction

Table 4: Response Tracker internal signals

5 Functional Behaviour

5.1 Allocation Flow

1. Tag Allocator asserts `TAG_VALID` with `TXN_TAG[7:0]` and `AWLEN[7:0]`.
2. Alloc Write Logic computes `exp_pkts = AWLEN + 1`.
3. Register File is written at address `TXN_TAG[7:0]`: `exp_pkts` stored, `cmp_pkts = 0`, `err_sticky = 0`.
4. If this is the first in-flight transaction for this AXID (occupancy 0→1), Head Pointer Array is initialised with `TXN_TAG[7:0]`.

5.2 Completion Flow

1. MC Core asserts `RESP_VALID` with `resp_type == 2'b01` (`WR_ACK`) and `tag[7:0]`.
2. Completion Update Logic asserts `inc_en` at address `tag[7:0]`: `cmp_pkts` incremented by 1.

3. If `resp_type == 2'b10 (ERR)`: additionally sets `err_sticky = 1` and latches `status[3:0]`.
4. Compare Logic evaluates `cmp_pkts == exp_pkts` combinationaly; asserts `txn_done` if true.

5.3 Emit Flow

1. `txn_done` pulses for a completing transaction.
2. Emit Logic checks whether the completing tag matches `head_tag` for that AXID.
3. If match (in-order): reads `err_sticky` from Register File, drives `BRESP_VALID`, `BID`, and `BRESP` (OKAY or DECERR) to B Response Generator. Asserts `advance_head` and `emit_done`.
4. If no match (out-of-order): transaction is complete in the Register File but the head has not yet completed. Emit Logic holds. `BRESP` is deferred until the head slot completes.

5.4 Free Flow

1. `emit_done` triggers Free Logic in the same cycle as `BRESP` emission.
2. Free Logic asserts `clear_en` to Register File: entry is invalidated.
3. `WR_FREE_VALID` and `WR_FREE_TAG` are asserted to Tag Allocator, decrementing occupancy for the freed AXID.

6 Timing Notes

- Alloc Write Logic is registered: Register File write takes effect the cycle after `TAG_VALID`.
- Completion Update Logic is registered: `cmp_pkts` increment takes effect the cycle after `RESP_VALID`.
- Compare Logic is combinational: `txn_done` appears in the same cycle the increment is visible — one cycle after the final `RESP_VALID`.
- Emit Logic is registered: `BRESP_VALID` is asserted the cycle after `txn_done` is evaluated.
- Free Logic fires in the same cycle as `BRESP` emission: `clear_en` and `WR_FREE_VALID` are asserted without additional latency.
- Simultaneous allocation and completion on the same tag are not possible by construction — a tag is only re-allocated after its slot is freed, which occurs after `BRESP` emission.

7 Edge Cases

- **Single-packet transaction (`AWLEN = 0`):** `exp_pkts = 1`. `txn_done` fires on the first and only `WR_ACK`. Handled identically to multi-packet transactions.
- **Partial error in a burst:** If one packet in a burst returns `ERR` and others return `WR_ACK`, `err_sticky` is set on the first error. All subsequent completions still increment `cmp_pkts` normally. On `BRESP` emission, `BRESP = DECERR` regardless of how many packets succeeded.
- **Out-of-order completion within an AXID:** AXI4 requires B channel responses to be in-order per AXID. If a later transaction of an AXID completes before the head transaction, Emit Logic holds the `BRESP` until the head slot is done. The later transaction's completion is safely held in the Register File.

- **RD_DATA completions:** Ignored entirely by Completion Update Logic. These are processed by the ROB and RTT on the read path.
- **Over-completion protection:** `cmp_pkts` must never exceed `exp_pkts`. Guaranteed by construction since MC Core returns exactly one `WR_ACK` per issued write packet. A design-time assertion should be included in the verification environment.
- **Simultaneous alloc and completion on same AXID (different seqnums):** These target different register file entries (different `tag[7:0]` addresses) — no conflict.